

```
@tf.function
def add(a, b):
    return (a+b)

print(add(tf.ones([2,2]), tf.ones([2,2])))
```

```
test-pc:~$ python3 tf-test.py
2.0.0-beta1
tf.Tensor(
[[2. 2.]
 [2. 2.]], shape=(2, 2), dtype=float32)
```

Here is another example to demonstrate automatic control flows and Autograph in action. Autograph automatically converts the conditions, while it loops Python to TensorFlow operators.

```
test-pc:~$ cat tf-test-control.py
import tensorflow as tf
```

```
print(tf.__version__)
```

```
@tf.function
def f(x):
    while tf.reduce_sum(x) > 1:
        x = tf.tanh(x)
    return x
```

```
print(f(tf.random.uniform([10])))
```

```
test-pc:~$ python3 tf-test-control.py
```

```
2.0.0-beta1
tf.Tensor(
[0.10785562 0.11102211 0.11347286 0.11239681 0.03989326
 0.10335539
 0.11030331 0.1135259 0.11357211 0.07324989], shape=(10,),
dtype=float32)
```

We can see Autograph in action with the following API over the function.

```
print(tf.autograph.to_code(f)) # f is the function name
```

TensorFlow Lite

The latest advancements in edge devices add neural network accelerators. Google has released EdgeTPU, Intel has the edge inference platform Movidius, Huawei mobile devices have the Kirin based NPU, Qualcomm has come up with NPE SDK to accelerate on the Snapdragon chipsets using Hexagon power and, recently, Samsung released Exynos 9 with NPU. An edge device optimised framework is necessary to support these hardware ecosystems.

Unlike TensorFlow, which is widely used in high power-consuming server infrastructure, edge devices are challenging in terms of reduced computing power, limited memory and battery constraints. TensorFlow Lite is aimed at bringing in TensorFlow models directly onto the edge with minimal effort. The TF Lite model format is different from TensorFlow. A TF Lite converter is available to convert a TensorFlow SavedBundle to a TF Lite model.

Though TensorFlow Lite is evolving, there are limitations too, such as in the number of operations supported, and the unsupported semantics like control-flows and RNNs. In its early days, TF Lite used a TOCO converter and there were a few challenges for the developer community. A brand new 2.0 converter is planned to be released soon. There are claims that using TF Lite results in huge improvements across the CPU, GPU and TPU.

TF Lite introduces delegates to accelerate parts of the graph on an accelerator. We may choose a specific delegate for a specific sub-graph, if needed.

```
#import "tensorflow/lite/delegates/gpu/metal_delegate.h"
```

```
// Initialize interpreter with GPU delegate
std::unique_ptr<Interpreter> interpreter;
InterpreterBuilder(*model, resolver)(&interpreter);
auto* delegate = NewGpuDelegate(nullptr); // default config
if (interpreter->ModifyGraphWithDelegate(delegate) !=
kTfLiteOk) return false;
```

```
// Run inference
while (true) {
    WriteToInputTensor(interpreter->typed_input_
tensor<float>(0));
    if (interpreter->Invoke() != kTfLiteOk) return false;
    ReadFromOutputTensor(interpreter->typed_output_
tensor<float>(0));
}
```

```
// Clean up
interpreter = nullptr;
DeleteGpuDelegate(delegate);
```

As shown above, we can choose GPUDelegate, and modify the graph with the respective kernel's runtime. TF Lite is going to support the Android NNAPI delegate, in order to support all the hardware that is supported by NNAPI. For edge devices, CPU optimisation is also important, as not all edge devices are equipped with accelerators; hence, there is a plan to support further optimisations for ARM and x86.

Optimisations based on quantisation and pruning are evolving to reduce the size and processing demands of models. Quantisation generally can reduce model size by 4x (i.e., 32-bit to 8-bit). Models with more convolution